

HyperProve: Answer-Guided Hypergraph Expansion for Multi-Hop Question Answering

Anonymous EMNLP submission

Abstract

Multi-hop question answering often fails when retrieval treats evidence as isolated matches to the original question, since the facts needed to answer a complex question are usually connected through intermediate entities, relations, and constraints. We propose **HyperProve**, a retrieval-augmented QA framework that addresses this challenge by coupling question decomposition with answer-conditioned expansion over a hypergraph of atomic facts. HyperProve does not use atomic facts, hypergraphs, or iterative retrieval in isolation; instead, it carries intermediate answers and supporting hyperedges as retrieval state, then uses that state to bias the next local hypergraph expansion. This design enables HyperProve to construct coherent evidence chains for final answer generation while making the retrieval process stateful and fact-centered. Across multi-hop QA benchmarks, HyperProve achieves the best overall performance in our evaluation, outperforming the strongest baselines by an average relative improvement of 6.2% in answer accuracy and 4.9% in F1.

1 Introduction

Retrieval-augmented generation (Lewis et al., 2020) has become a standard way to ground language models in external evidence, but many retrieval pipelines still treat question answering as a one-shot relevance problem: given a question, retrieve passages that are independently similar to it, then ask a reader to generate the answer. This setup is better suited to questions whose answer-bearing evidence is directly aligned with the query. In multi-hop question answering, however, the crucial evidence is often distributed across facts connected through intermediate entities, relations, temporal constraints, or other bridge conditions (Trivedi et al., 2022; Yang et al., 2018; Ho et al., 2020). In such cases, the next useful fact may not be similar to the original question at all; it becomes relevant

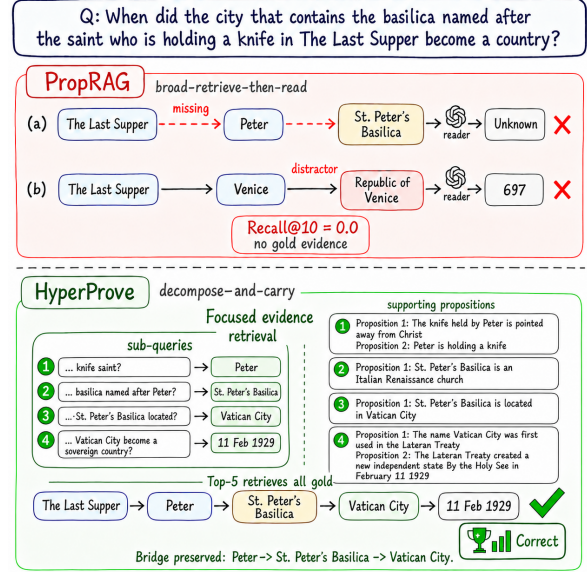


Figure 1: Case study of a 4-hop question: PropRAG fails to connect *Last Supper* to *Peter* and drifts to the distractor chain *Venice* $\rightarrow$ *Republic of Venice*; HyperProve succeeds because atomic-fact hyperedges explicitly preserve the bridge entities and correct path.

only after an earlier fact has established the right bridge or connection.

Prior work attacks this problem through graph structure and iterative retrieval, but each leaves a concrete failure mode for multi-hop QA. Graph-based and structured RAG methods connect passages, entities, or atomic facts to move beyond independent dense similarity (Guo et al., 2025; Gutiérrez et al., 2025; Zhu et al., 2025b). However, passage-level nodes often mix several facts, entities, and distractors in one unit, so the graph can retrieve a relevant passage without identifying which fact should be followed next. Binary triples have the opposite problem: they make retrieval fine-grained, but can split one textual fact into disconnected pairwise edges and drop qualifiers, provenance, or multi-entity context needed to preserve the correct bridge (Wang and Han, 2025).

As shown in Figure 1, one missed bridge can push PropRAG (Wang and Han, 2025), a prior structured retrieval method, toward a distractor chain rather than the evidence needed for the correct answer.

Iterative retrieval methods decompose questions, rewrite queries, or interleave retrieval with reasoning (Trivedi et al., 2023; Ye et al., 2025; Zhu et al., 2025a; Wu et al., 2025). These methods adapt the query across hops, but the state passed between hops is often textual: an intermediate answer is inserted into the next query, while the retrieval system is not necessarily forced to continue from the evidence unit that supported that answer. As a result, the next hop can restart from surface similarity, lose the bridge fact, or follow a plausible distractor entity.

We propose **HyperProve**, a retrieval-augmented QA framework that makes retrieval state explicit at the level of atomic-fact hyperedges. In contrast to methods that rank a fixed structured index or only pass textual state between hops, HyperProve carries the intermediate answer and its supporting hyperedges as an answer-conditioned frontier for the next hop. It decomposes a complex question into dependent sub-queries, rewrites later sub-queries with resolved bridge entities, and uses the carried frontier to seed query-biased local hypergraph expansion. This lets retrieval move from one grounded atomic fact to structurally connected candidate facts instead of restarting from the original question or expanding without answer-conditioned state. In Figure 1, for HyperProve, this frontier preserves the hidden bridge evidence and keeps the reasoning chain on the correct path.

Our main contributions are summarized as follows:

- We propose **HyperProve**, an answer-conditioned hyperedge retrieval formulation that unifies graph-based and iterative RAG: each intermediate answer is carried with its supporting atomic-fact hyperedges as the structural frontier for subsequent hops.
- We design a query-biased local hypergraph expansion algorithm that retrieves over atomic-fact hyperedges, combining structural connectivity among evidence units with semantic alignment to each rewritten sub-query.
- We empirically show that HyperProve improves multi-hop QA across three benchmarks, outperforming dense, structured, and

iterative retrieval baselines.

2 Related Work

Graph-based RAG. Graph-based RAG improves over dense retrieval by modeling relationships among passages, entities, or atomic facts instead of scoring passages independently. HippoRAG2 builds graph memory with Personalized PageRank (Gutiérrez et al., 2025); PropRAG searches proposition paths to retain fine-grained context (Wang and Han, 2025); HyperGraphRAG represents n -ary facts as hyperedges (Luo et al., 2025); and HGRAG uses an entity-passage hypergraph for multi-hop QA (Wang et al., 2026). Related hyperedge or hyper-relational methods also support this direction: HyperFM models hyper-relational KG facts (Lu et al., 2025), OG-RAG retrieves ontology-grounded factual hyperedges (Sharma et al., 2025), and HyperMem organizes long-term memory with hierarchical hyperedges (Yue et al., 2026). However, they remain suboptimal for multi-hop QA because they mainly rank fixed units, optimize compact evidence for one query, or traverse memory without anchoring the next hop to the evidence that produced the current bridge entity. HyperProve instead carries intermediate answers and supporting hyperedges across sub-queries, then uses them to guide query-biased local expansion over atomic facts rather than restarting retrieval from surface similarity.

Iterative Retrieval. Iterative retrieval methods use LLMs to plan, retrieve, and refine evidence across multiple steps, with IRCot interleaving retrieval and chain-of-thought reasoning so that intermediate reasoning guides later retrieval (Trivedi et al., 2023). T2RAG uses triplet-driven thinking to move beyond raw chunks (Gong et al., 2025). HopRAG performs retrieve-reason-prune steps over retrieved evidence (Liu et al., 2025), while ChainRAG mitigates lost-in-retrieval errors through progressive retrieval and query rewriting (Zhu et al., 2025a). More recent systems further adapt retrieval across hops: KiRAG performs knowledge-driven iterative retrieval over triples (Fang et al., 2025), and UniRAG combines decomposition, break-down reasoning, and iterative rewriting (Kim et al., 2025). However, when cross-hop state is mainly textual, later retrieval can drift from the supporting fact chain and restart from surface similarity. HyperProve shares the stateful view of these systems, but

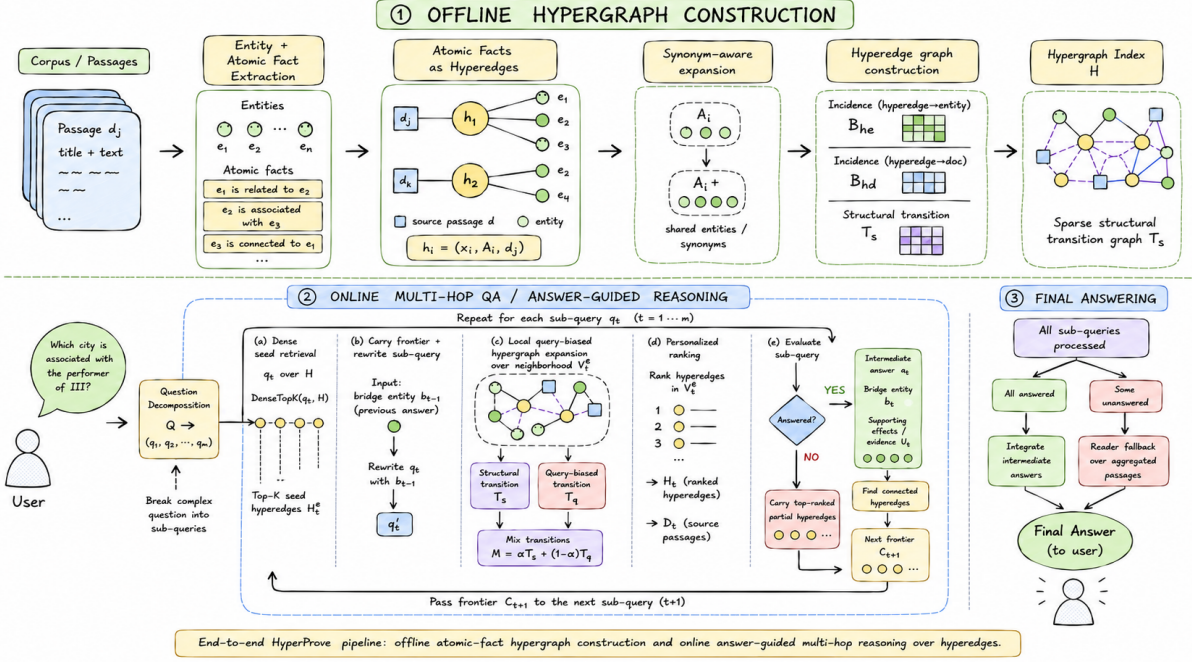


Figure 2: Overall HyperProve pipeline. The system builds an offline synonym-aware atomic-fact hypergraph from corpus passages, then performs online answer-guided multi-hop reasoning by decomposing a question into sub-queries, carrying intermediate answers as bridge entities, expanding over local hypergraph neighborhoods, and integrating the resulting evidence chain into a final answer.

makes retrieval state structural: intermediate answers update a frontier of atomic-fact hyperedges, and each subsequent step is constrained by hypergraph transitions while remaining biased toward the current sub-query.

3 Methodology

Given a corpus of passages $\mathcal{D} = \{d_j\}_{j=1}^{|\mathcal{D}|}$, our goal is to retrieve a compact set of connected evidence and generate an answer a for a question Q . We view a multi-hop question as a sequence of dependent sub-queries,

$$Q \rightarrow (q_1, q_2, \dots, q_m),$$

where each q_t resolves at least one bridge entity, bridge relation, or final target attribute. The answer or evidence from q_{t-1} conditions the interpretation and retrieval frontier of q_t . HyperProve operationalizes this view by constructing a hyperedge-level knowledge structure and performing all intermediate reasoning directly over atomic-fact hyperedges. Figure 2 summarizes the resulting offline indexing and online answer-guided reasoning pipeline.

3.1 Atomic Facts as Hyperedges

As the first offline stage in Figure 2, HyperProve extracts an entity set for each passage d_j ,

$$E_j = \{e_1, e_2, \dots, e_{m_j}\},$$

including named entities, dates, salient objects, and generic concepts that participate in meaningful relations. The passage is then decomposed into atomic facts. Each atomic fact is represented as a hyperedge

$$h_i = (x_i, A_i, d_j),$$

where x_i is a self-contained factual statement, $A_i \subseteq E_j$ is the set of participating entities, and d_j is the source passage. Structurally, the same object can be viewed as

$$h_i = A_i \cup \{d_j\}.$$

This modeling choice is motivated by the limitations of binary triples. A triple represents a fact as a pairwise relation, such as (s, r, o) , but many textual facts naturally involve more than two participants, together with qualifiers such as time, place, role, or provenance. Decomposing such a fact into several pairwise triples can cause *context collapse*: the local context that makes the fact meaningful is

fragmented or lost. PropRAG identifies this context loss in triple-based structured RAG and instead uses context-rich fact units to improve faithful multi-hop retrieval (Wang and Han, 2025). HyperProve follows the same motivation, but represents each extracted atomic fact as a hyperedge. This allows one factual unit to connect all participating entities and its source passage simultaneously. Prompts for entity and hyperedge extraction can be found in the Appendix.

3.2 Knowledge Graph Constructions

The next offline stage in Figure 2 turns the corpus into a synonym-aware hyperedge knowledge graph. Entity surface forms may differ across passages, so HyperProve embeds entity strings and creates a synonym map using cosine similarity. If two entity embeddings exceed a threshold η (set to 0.8 in our experiments), they are treated as neighboring entity mentions. For each hyperedge, the entity set is expanded as

$$A_i^+ = A_i \cup \text{Syn}(A_i).$$

The expanded set A_i^+ allows hyperedges to overlap even when passages use different surface forms for the same entity or concept.

Let N be the number of hyperedges, M_e the number of expanded entities, and M_d the number of passages. We define the hyperedge-entity incidence matrix $B_{he} \in \{0, 1\}^{N \times M_e}$ and the hyperedge-passage incidence matrix $B_{hd} \in \{0, 1\}^{N \times M_d}$:

$$\begin{aligned} B_{he}[i, a] &= \mathbf{1}[e_a \in A_i^+], \\ B_{hd}[i, j] &= \mathbf{1}[h_i \text{ originates from } d_j]. \end{aligned}$$

The corresponding degree matrices are defined as

$$\begin{aligned} D_h[i, i] &= \sum_a B_{he}[i, a] + \sum_j B_{hd}[i, j], \\ D_e[a, a] &= \sum_i B_{he}[i, a], \\ D_d[j, j] &= \sum_i B_{hd}[i, j]. \end{aligned}$$

The structural adjacency between hyperedges is then computed as

$$A_s = D_h^{-1} (B_{he} D_e^{-1} B_{he}^\top + B_{hd} D_d^{-1} B_{hd}^\top).$$

The term $B_{he} D_e^{-1} B_{he}^\top$ connects hyperedges that share expanded entities, while $B_{hd} D_d^{-1} B_{hd}^\top$ connects hyperedges originating from the same source

Algorithm 1 HyperProve main algorithm.

Require: Hyperedge index $(H, E, \mathcal{D}, T_s)$, question Q , seed split $K_c = K_d = 10$
Ensure: Final answer a
1: $(q_1, \dots, q_m) \leftarrow \text{DECOMPOSE}(Q)$
2: $\mathcal{M}_H \leftarrow \emptyset, \mathcal{M}_D \leftarrow \emptyset, \mathcal{C}_1 \leftarrow \emptyset, \mathcal{A} \leftarrow \emptyset$
 $\mathcal{M}_H$: scored hyperedge memory; $\mathcal{M}_D$: document memory
3: **for** $t = 1$ to m **do**
4: **if** $t = 1$ **then**
5: $S_t \leftarrow \text{DENSETOPK}(q_t, H, K_d)$
6: **else**
7: $q'_t \leftarrow \text{REWRITE}(q_t, \mathcal{A})$
8: $S_t \leftarrow \text{TOPK}(\mathcal{C}_t, K_c) \cup \text{DENSETOPK}(q'_t, H, K_d)$
9: **end if**
10: retrieve H_t , scores π_t , and D_t from S_t
11: $\mathcal{M}_H \leftarrow \mathcal{M}_H \cup \{(H_t, \pi_t)\}$
12: $\mathcal{M}_D \leftarrow \text{MAXMERGE}(\mathcal{M}_D, D_t)$
13: $(s_t, a_t, U_t) \leftarrow \text{EVALUATE}(q_t, H_t, D_t)$
 s_t : answer status; a_t : sub-answer; U_t : supporting hyperedges
14: **if** $s_t = \text{ANSWERED}$ **then**
15: $\mathcal{A} \leftarrow \mathcal{A} \cup \{(q_t, a_t)\}$
16: $b_t \leftarrow \text{BRIDGEENTITY}(a_t)$
17: $E_t \leftarrow \text{ENTITIES}(U_t)$
18: $\mathcal{C}_{t+1} \leftarrow \text{FINDHYPEREDGES}(b_t \cup E_t, H)$
19: **else**
20: $\mathcal{C}_{t+1} \leftarrow \text{TOPK}(H_t)$
21: **end if**
22: **end for**
23: **if** all sub-queries are answered **then**
24: $a \leftarrow \text{INTEGRATEANSWERS}(Q, \mathcal{A})$
25: **else**
26: $\mathcal{P}_k \leftarrow \text{TOPKPASSAGES}(\mathcal{M}_H, \mathcal{M}_D)$
27: $a \leftarrow \text{READERFALLBACK}(Q, \mathcal{P}_k, \mathcal{M}_H)$
28: **end if**
29: **return** a

passage. The inverse degree factors down-weight overly common entities, large passages, and hyperedges with many endpoints. We remove self-loops and row-normalize:

$$T_s = \text{RowNormalize}(A_s - \text{diag}(A_s)).$$

The resulting T_s is a sparse structural transition matrix over atomic-fact hyperedges. Importantly, this graph is not used to reason over passages directly. Passages remain provenance and reader context, while the reasoning graph itself is defined over hyperedges.

3.3 Query-biased Hyperedge Retrieval

For each query or sub-query, HyperProve uses dense retrieval only to select entry hyperedges. These seeds define where graph search begins; evidence is then ranked by navigating connected atomic facts in the hyperedge graph.

Starting from the current seeds, HyperProve builds a local subgraph G_t^* by random-walk expansion over the structural transition matrix T_s , retaining at most 600 hyperedge nodes connected through shared entities, synonym-expanded entity overlap, or source-passage connectivity. Let T_s^* be the row-normalized restriction of T_s to G_t^* . Within this subgraph, HyperProve constructs a query-biased transition matrix T_q^* . Let $c_j =$

$\cos(\phi(q'_t), \phi(x_j))$ be the similarity between the rewritten sub-query q'_t and hyperedge statement x_j , and let $\mathcal{N}_\theta(i) = \{k \in \mathcal{N}(i) : c_k > \theta\}$. With temperature τ , it is

$$T_q^*[i, j] = \begin{cases} \frac{\exp(c_j/\tau)}{\sum_{k \in \mathcal{N}_\theta(i)} \exp(c_k/\tau)}, & j \in \mathcal{N}_\theta(i), \\ 0, & \text{otherwise,} \end{cases}$$

where $\mathcal{N}(i)$ is the structural neighborhood of hyperedge h_i in G_t^* . Thus, each row distributes probability only among structurally connected hyperedges whose query similarity exceeds θ ; if no such neighbor exists, the query-biased row is zero and retrieval relies on the structural transition.

The final transition matrix mixes structural connectivity and query relevance:

$$M = \alpha T_s^* + (1 - \alpha) T_q^*.$$

Here, α controls how much the walk follows the hypergraph structure versus the current query semantics. Personalized ranking is then performed over this local hyperedge graph:

$$\pi_{r+1} = (1 - \gamma)u^* + \gamma M^\top \pi_r,$$

where u^* is the reset distribution over the current seed hyperedges. We use $\tau = 1$, $\theta = 0.4$, $\alpha = 0.5$, and $\gamma = 0.85$ in all experiments. These hyperparameters were not optimized per dataset; and tuning them is a likely source of further performance improvement. The resulting score vector ranks atomic-fact hyperedges, not passages. Passages are only recovered after hyperedge ranking, by selecting those that contain highly ranked hyperedges. This design successfully makes the retrieval process explicitly fact-centered.

3.4 Answer-Guided Hyperedge Reasoning

HyperProve models multi-hop question answering as the online answer-guided reasoning stage in Figure 2. It decomposes a question into dependent sub-queries and carries evidence across them as a hyperedge frontier; Algorithm 1 gives the full procedure. Given $Q \rightarrow (q_1, q_2, \dots, q_m)$ and the corpus hyperedges $\mathcal{H} = \{h_i\}_{i=1}^N$, let $\mathcal{C}_1 = \emptyset$, and for $t > 1$, let $\mathcal{C}_t \subseteq \mathcal{H}$ denote the carried frontier produced after hop $t - 1$. HyperProve uses at most 20 seeds per hop, split evenly between $K_c = 10$ carried-frontier seeds and $K_d = 10$ fresh dense seeds retrieved with the rewritten query. The dense seeds are retained in later hops so retrieval can escape an overly local hyperedge neighborhood when

the next supporting fact may not be reachable from the carried frontier alone. The seed set for hop t is

$$\mathcal{D}_t = \text{DenseTopK}(q'_t, \mathcal{H}, K_d),$$

$$\mathcal{S}_t = \begin{cases} \mathcal{D}_t, & t = 1, \\ \text{TopK}(\mathcal{C}_t, K_c) \cup \mathcal{D}_t, & t > 1. \end{cases}$$

Here, $\mathcal{C}_t$ is the carried frontier used at hop t , and q'_t is the retrieval query after optional rewriting. For the first hop, $q'_t = q_1$. For $t > 1$, $q'_t = \text{Rewrite}(q_t, \{(q_s, a_s)\}_{s < t})$, which replaces referring expressions with previous intermediate answers while preserving the original intent. If no reliable previous answer is available, HyperProve uses $q'_t = q_t$.

When a sub-query is answered, the predicted bridge entity b_t and the entities in the supporting atomic facts U_t define the next carried frontier:

$$\mathcal{C}_{t+1} = \text{FindHyperedges}(\{b_t\} \cup \text{Entities}(U_t), \mathcal{H}),$$

However, if a sub-query cannot be fully resolved, HyperProve carries the top-ranked hyperedges after PPR as partial evidence for the next iteration.

After all sub-queries are processed, HyperProve either integrates the intermediate answers or falls back to an LLM reader over $\text{TopKPassages}(\mathcal{M}_H, \mathcal{M}_D)$. Given the scored hyperedge memory $\mathcal{M}_H = \{(H_t, \pi_t)\}_{t=1}^m$, each passage is scored by its strongest retrieved hyperedge:

$$s(d) = \max_{\substack{t \in [m], h \in H_t \\ \text{src}(h)=d}} \pi_t(h),$$

where $\text{src}(h)$ denotes the source passage of hyperedge h . Thus, TopKPassages returns the top- k passages, with $k = 10$ in our experiments, under $s(d)$, while $\mathcal{M}_H$ retains the hop-level evidence.

4 Experiments

4.1 Experimental Settings

Datasets and Metrics. We evaluate HyperProve on three open-domain multi-hop question answering benchmarks: MuSiQue (Trivedi et al., 2022), HotpotQA (Yang et al., 2018), and 2WikiMultiHopQA (Ho et al., 2020). We report three answer-quality metrics: LLM-as-a-judge answer accuracy, Exact Match (EM), and token-level F1.

Baselines. We compare HyperProve against three families of retrieval-augmented baselines. First, the dense retrieval family includes Contriever (Izacard et al., 2022) and BGE-M3 (Chen et al.,

Table 1: Main results on multi-hop QA benchmarks. We report LLM-as-a-judge answer accuracy, Exact Match (EM), and token-level F1. Best results are in **bold**; second-best results are underlined.

Method	MuSiQue			HotpotQA			2Wiki		
	Acc.	EM	F1	Acc.	EM	F1	Acc.	EM	F1
<i>Dense Retrieval</i>									
Contriever	47.80	22.30	32.10	55.55	44.40	57.50	69.60	38.50	43.70
BGE-M3	51.50	27.80	39.70	68.12	47.80	61.80	72.18	48.20	54.50
<i>StructuredKG</i>									
HippoRAG2	52.00	37.20	48.60	81.70	62.70	<u>75.50</u>	86.40	65.00	71.00
PropRAG	50.90	29.30	46.73	<u>85.80</u>	57.80	73.79	77.50	58.00	68.78
HyperGraphRAG	55.13	42.20	53.80	77.89	<u>63.90</u>	73.80	77.72	<u>68.30</u>	71.30
HGRAG	58.60	41.19	<u>55.04</u>	80.10	54.90	69.43	85.00	64.10	74.10
<i>Iterative Retrieval</i>									
IRCoT	56.67	34.10	47.60	85.23	55.70	71.20	83.65	60.70	74.30
T2RAG	54.67	34.30	45.60	81.33	54.20	67.30	83.75	66.70	74.40
HopRAG	<u>67.18</u>	<u>42.40</u>	54.90	85.51	61.10	68.26	<u>86.66</u>	62.00	<u>76.06</u>
ChainRAG	60.01	41.20	54.15	85.46	60.70	74.66	82.48	63.30	74.39
HyperProve	71.60	42.83	59.23	91.30	63.92	76.64	91.40	69.70	80.01

2024). Second, the StructuredKG family includes graph- and structure-based RAG methods: HippoRAG2 (Gutiérrez et al., 2025), PropRAG (Wang and Han, 2025), HyperGraphRAG (Luo et al., 2025), and HGRAG (Wang et al., 2026). Third, the iterative retrieval family includes IRCoT (Trivedi et al., 2023), T2RAG (Gong et al., 2025), HopRAG (Liu et al., 2025), and ChainRAG (Zhu et al., 2025a).

Implementation Details. For all methods in the main experiments and analysis, we use BGE-M3 as the retriever and GPT-OSS-20B as the backbone LLM. Apart from this shared model setup, each baseline follows its original implementation, including its hyperparameter configurations. We report results averaged over three runs to reduce variance and ensure a consistent comparison across methods.

4.2 Main Results

Dense retrieval lacks evidence state for multi-hop QA. Although BGE-M3 improves over Contriever, it remains far behind HyperProve: on MuSiQue, 51.50 accuracy and 39.70 F1 versus 71.60 and 59.23; on 2Wiki, 72.18 and 54.50 versus 91.40 and 80.01. Even on HotpotQA, where dense retrieval is stronger, BGE-M3 is still 23.18 accuracy points lower. These gaps show that independent dense matching can retrieve locally rele-

vant passages, but lacks persistent evidence state for later hops once a bridge entity or constraint has been resolved.

Static structure and iterative retrieval are complementary but incomplete. The strongest baseline alternates between StructuredKG and Iterative Retrieval methods: HopRAG leads accuracy on MuSiQue with 67.18 and 2Wiki with 86.66, while PropRAG leads HotpotQA accuracy with 85.80; HGRAG gives the strongest MuSiQue F1 with 55.04, and HyperGraphRAG gives the strongest baseline EM on HotpotQA with 63.90 and 2Wiki with 68.30. This uneven pattern reflects the limits of both families. Static structure helps connect evidence but uses a mostly fixed frontier that is not conditioned on intermediate answers, while iterative retrieval adapts the query across hops but can drift from the evidence that justified the previous hop.

HyperProve achieves the best evaluated performance. HyperProve ranks first on every reported metric across all three benchmarks. It improves answer accuracy over HopRAG by 4.42 points on MuSiQue and 4.74 points on 2Wiki; on HotpotQA, it improves over PropRAG by 5.50 accuracy points and over HippoRAG2 by 1.14 F1 points. The gains are consistent across accuracy, EM, and F1, suggesting that HyperProve improves both final an-

Variant	Overall	2-hop	3-hop	4-hop
HyperProve	71.60	78.00	72.96	49.40
w/o bridge entity	64.50	69.11	67.41	44.58
w/o fresh dense seed	61.30	71.24	59.18	34.34
w/o decomposition	51.28	58.11	49.05	33.73
w/o T_s	62.13	67.02	64.65	42.06
w/o T_q	58.17	65.20	58.60	35.40

Table 2: Accuracy breakdown of HyperProve ablations on MuSiQue by reasoning depth. Accuracy is computed using judge verdicts over 1000 questions: 518 2-hop, 316 3-hop, and 166 4-hop examples.

swers and evidence coverage. The results support HyperProve’s design: intermediate answers should define a structural frontier over atomic-fact hyperedges, not only rewrite later queries.

5 Analysis

We analyze HyperProve from four perspectives: component ablations, evidence-chain faithfulness, efficiency, and qualitative behavior. Together, these analyses examine which mechanisms drive multi-hop performance, whether it accumulates connected evidence across hops, whether HyperProve improves accuracy with reasonable cost, and how it handles hidden bridge entities and multi-entity reasoning.

5.1 Retrieval Component Breakdown

The answer-guided ablations in Table 2 show that HyperProve relies on both bridge propagation and fresh query-specific entry points. Removing bridge entities lowers overall accuracy from 71.60 to 64.50, with a substantial drop on 4-hop questions from 49.40 to 44.58, suggesting that intermediate answers provide cross-hop links that dense similarity alone may miss. Removing fresh dense seeds is also especially harmful for deeper reasoning: although 2-hop accuracy remains comparable, 4-hop accuracy falls to 34.34, indicating that the carried frontier alone can trap retrieval inside the current hyperedge neighborhood. The largest degradation comes from removing decomposition, which reduces overall accuracy to 51.28 and confirms that complex questions are better solved through dependent sub-queries than through a single retrieval step.

The transition-component ablations show that both T_s and T_q are necessary for effective hyper-edge reasoning. Removing the structural transition T_s produces a pattern similar to removing bridge entities, but with a larger overall drop to 62.13, in-

Method	Recall@10			Avg Gain
	MuSiQue	HotpotQA	2Wiki	After Hop 1
HyperProve	82.31	96.20	98.15	+41.24
ChainRAG	67.42	89.45	75.85	+26.35
HopRAG	66.63	82.82	84.17	+25.98

Table 3: Evidence chain faithfulness across multi-hop benchmarks. Dataset columns report Recall@10, computed as the average fraction of gold passages retrieved in the top 10 results. Avg Gain after Hop 1 measures the average increase in cumulative gold-passage coverage after the first hop.

dicating that structural connectivity is crucial for maintaining valid evidence paths across hops. Removing the query-conditioned transition T_q causes an even sharper decline to 58.17 overall and 35.40 on 4-hop questions. This shows that structure alone is not enough: without query-aware transition, HyperProve no longer performs targeted graph reasoning and may expand along connected but irrelevant hyperedges.

5.2 Evidence Chain Faithfulness

Table 3 evaluates evidence quality under the same top-10 retrieval budget across MuSiQue, HotpotQA, and 2Wiki, rather than only reporting answer accuracy. HyperProve achieves the highest Recall@10 on all three benchmarks, with 82.31 on MuSiQue, 96.20 on HotpotQA, and 98.15 on 2Wiki. These results improve over the strongest iterative baselines by 14.89, 6.75, and 13.98 points, respectively. HyperProve also obtains the largest average gain after the first hop, +41.24 versus +26.35 for ChainRAG and +25.98 for HopRAG. Thus, additional hops are not simply adding more context: they recover missing gold passages while preserving a stronger final support set than both iterative baselines.

Figure 3 further isolates the hop-wise behavior of HyperProve. Across all three benchmarks, each decomposed hop increases or maintains cumulative support coverage, with especially large gains after the first hop. Because the curve measures accumulated gold-passage coverage at Recall@10, these increases reflect newly recovered supporting evidence rather than longer reasoning traces or repeated retrievals.

5.3 Efficiency Analysis

Table 5 shows that HyperProve offers a worthwhile cost-latency tradeoff for stronger multi-hop reasoning. While it is more expensive than dense retrieval

System	Top-10 Coverage	Retrieval / Reasoning Behavior	Final Answer
HyperProve	4 / 4	Separately resolves <i>Pathology</i> → <i>San Diego</i> and <i>Still Swingin'</i> → <i>Papa Roach</i> → <i>California</i> , then applies the rank query to <i>San Diego</i> under the <i>California</i> urban-area constraint.	third-largest
ChainRAG	2 / 4	Retrieves evidence for <i>Pathology</i> → <i>San Diego</i> and <i>Still Swingin'</i> → <i>Papa Roach</i> , but misses the facts needed to bind <i>Papa Roach</i> → <i>Vacaville, California</i> and <i>San Diego urban area</i> → <i>third-largest in California</i> . Its reader is then pulled to a nearby city-rank answer.	2nd

Table 4: Qualitative comparison on a multi-entity MuSiQue example. HyperProve retrieves the complete evidence chain and preserves the urban-area constraint, while ChainRAG retrieves only partial evidence and confuses city population rank with urban-area rank.

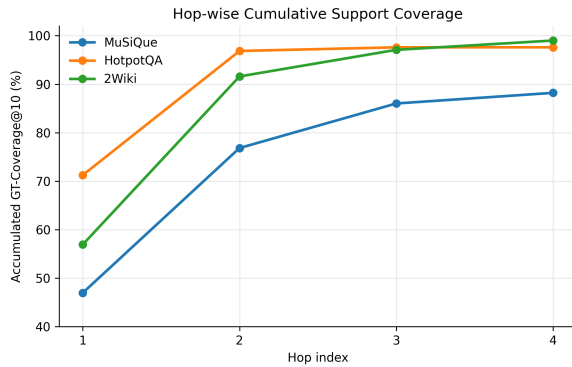


Figure 3: Hop-wise cumulative support coverage. Cumulative gold-passage coverage increases across hops, showing that later sub-queries progressively recover missing support evidence.

Method	Acc.	# Tokens	Lat. (s)
BGE-M3	51.50	1.7M	4.1K
IRCoT	56.67	4.4M	3.8K
T2RAG	54.67	2.4M	9.6K
HopRAG	67.18	36.1M	45.2K
ChainRAG	60.01	15.4M	37.0K
HyperProve	71.60	9.6M	31.9K

Table 5: Efficiency comparison on MuSiQue. Accuracy is LLM-as-a-Judge accuracy. Cost statistics exclude judge costs.

and lightweight iterative retrieval baselines, the accuracy gains are substantial: HyperProve improves over BGE-M3 by 20.10 points, IRCoT by 14.93 points, and T2RAG by 16.93 points. More importantly, compared with stronger multi-hop baselines, HyperProve achieves better accuracy with lower cost: it outperforms HopRAG by 4.42 points while using only 9.6M tokens instead of 36.1M and reducing latency from 45.2K to 31.9K seconds. It also improves over ChainRAG by 11.59 points with fewer tokens and lower latency. These results suggest that HyperProve’s answer-guided hypergraph expansion adds computation where it matters, yield-

ing higher accuracy without the excessive cost of unconstrained iterative retrieval.

5.4 Qualitative Analysis

Table 4 presents a multi-entity MuSiQue case that requires linking *Pathology* to *San Diego* and *Still Swingin'* to *Papa Roach* and *California*, then applying the rank constraint to the San Diego urban area within California. HyperProve retrieves all four gold passages and keeps entities, constraints, and provenance bound within atomic hyperedges, allowing it to preserve both chains and correctly answer *third-largest*.

In contrast, ChainRAG recovers only the partial links *Pathology* → *San Diego* and *Still Swingin'* → *Papa Roach*, but misses the facts binding *Papa Roach* to *Vacaville, California* and San Diego to the *third-largest* urban area in California. Its reader is therefore pulled toward the nearby but incorrect city-population rank, answering *2nd*. The case shows that HyperProve’s answer-guided hypergraph expansion helps preserve not just more evidence, but also the correct entity scope and comparison constraint.

6 Conclusion

We presented HyperProve, an answer-guided hypergraph expansion framework. HyperProve decomposes complex questions into dependent sub-queries, carries intermediate answers and supporting hyperedges as retrieval state, and performs query-biased local expansion over atomic-fact hyperedges that preserve multi-entity context and source provenance. Across three multi-hop QA benchmarks, HyperProve achieves the strongest performance among dense, structured, and iterative retrieval baselines. These results show the value of combining stateful query control with structured evidence accumulation.

Limitations

HyperProve has two limitations that reflect broader open challenges for multi-hop RAG. First, our evaluation relies partly on LLM-as-a-judge accuracy, which may miss subtle reasoning errors, evidence insufficiency, or faithfulness issues; future work should include human evaluation and stronger evidence-chain metrics. Second, like the baselines, HyperProve is tested mainly on fixed open-domain QA benchmarks with stable corpora and predefined answers. This setting does not fully capture noisy, incomplete, conflicting, multilingual, or temporally changing evidence, which remains an important direction for future evaluation.

References

Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. [M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2318–2335, Bangkok, Thailand. Association for Computational Linguistics.

Jinyuan Fang, Zaiqiao Meng, and Craig Macdonald. 2025. [KiRAG: Knowledge-driven iterative retriever for enhancing retrieval-augmented generation](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 18969–18985. Association for Computational Linguistics.

Shengbo Gong, Xianfeng Tang, Carl Yang, and Wei Jin. 2025. [Beyond chunks and graphs: Retrieval-augmented generation through triplet-driven thinking](#). Preprint, arXiv:2508.02435.

Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. [LightRAG: Simple and fast retrieval-augmented generation](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10746–10761. Association for Computational Linguistics.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. [From RAG to memory: Non-parametric continual learning for large language models](#). In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 21497–21515. PMLR.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. [Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625. International Committee on Computational Linguistics.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. [Unsupervised dense information retrieval with contrastive learning](#). *Transactions on Machine Learning Research*.

Gun Il Kim, Jong Wook Kim, and Beakcheol Jang. 2025. [UniRAG: A unified RAG framework for knowledge-intensive queries with decomposition, break-down reasoning, and iterative rewriting](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 18795–18810, Suzhou, China. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive NLP tasks](#). In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, Virtual*.

Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. 2025. [HopRAG: Multi-hop reasoning for logic-aware retrieval-augmented generation](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 1897–1913. Association for Computational Linguistics.

Yuhuan Lu, Weijian Yu, Xin Jing, and Dingqi Yang. 2025. [HyperFM: Fact-centric multimodal fusion for link prediction over hyper-relational knowledge graphs](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 2818–2830. Association for Computational Linguistics.

Haoran Luo, Haihong E, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, and Anh Tuan Luu. 2025. [HyperGraphRAG: Retrieval-augmented generation via hypergraph-structured knowledge representation](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 152206–152234. Curran Associates, Inc.

Kartik Sharma, Peeyush Kumar, and Yunqing Li. 2025. [OG-RAG: Ontology-grounded retrieval-augmented generation for large language models](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 32962–32981. Association for Computational Linguistics.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. [MuSiQue: Multi-hop questions via single-hop question composition](#). *Transactions of the Association for Computational Linguistics*, 10:539–554.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. [Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions](#). In *Proceedings of the*

649	61st Annual Meeting of the Association for Computational Linguistics, pages 10014–10037. Association for Computational Linguistics.
650	
651	
652	Changjian Wang, Weihong Deng, Weili Guan, Quan Lu, and Ning Jiang. 2026. Cross-granularity hypergraph retrieval-augmented generation for multi-hop question answering . <i>Proceedings of the AAAI Conference on Artificial Intelligence</i> , 40(39):33368–33376.
653	
654	
655	
656	
657	Jingjin Wang and Jiawei Han. 2025. PropRAG: Guiding retrieval with beam search over proposition paths . In <i>Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing</i> , pages 6212–6227, Suzhou, China. Association for Computational Linguistics.
658	
659	
660	
661	
662	
663	Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. 2025. Agentic reasoning: A streamlined framework for enhancing LLM reasoning with agentic tools . In <i>Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics</i> , pages 28489–28503. Association for Computational Linguistics.
664	
665	
666	
667	
668	
669	
670	Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering . In <i>Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing</i> , pages 2369–2380. Association for Computational Linguistics.
671	
672	
673	
674	
675	
676	
677	
678	Linhao Ye, Lang Yu, Zhikai Lei, Qin Chen, Jie Zhou, and Liang He. 2025. Optimizing question semantic space for dynamic retrieval-augmented multi-hop question answering . In <i>Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics</i> , pages 17814–17824. Association for Computational Linguistics.
679	
680	
681	
682	
683	
684	
685	Juwei Yue, Chuanrui Hu, Jiawei Sheng, Zuyi Zhou, Wenyan Zhang, Tingwen Liu, Li Guo, and Yafeng Deng. 2026. HyperMem: Hypergraph memory for long-term conversations . <i>Preprint</i> , arXiv:2604.08256.
686	
687	
688	
689	
690	Rongzhi Zhu, Xiangyu Liu, Zequn Sun, Yiwei Wang, and Wei Hu. 2025a. Mitigating lost-in-retrieval problems in retrieval augmented multi-hop question answering . In <i>Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics</i> , pages 22362–22375. Association for Computational Linguistics.
691	
692	
693	
694	
695	
696	
697	Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yaliang Li, and Wei Hu. 2025b. Knowledge graph-guided retrieval augmented generation . In <i>Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies</i> , pages 8912–8924. Association for Computational Linguistics.
698	
699	
700	
701	
702	
703	

A Appendix

This appendix provides additional details for HyperProve, including the prompts used for offline hypergraph construction and online answer-guided reasoning, dataset statistics, positioning relative to prior work, and an additional case study.

A.1 Prompt Templates

This section lists the prompt templates used by HyperProve for offline hypergraph construction, online answer-guided retrieval, answer generation, and automatic evaluation. All prompts are designed to produce structured outputs, which simplifies downstream parsing and makes the reasoning pipeline more reproducible.

A.1.1 Entity Extraction

Task:

Given a passage, identify the entities that are needed to represent the factual content of the passage.

Output format:

Return only a valid JSON object with the following structure:

```
{
  "entities": ["entity_1", "entity_2", "..."]
}
```

Guidelines:

1. Include all named entities mentioned in the passage.
2. Include all dates, temporal expressions, and events.
3. Include generic concepts when they are central to the passage topic.
4. Include entities that participate in meaningful relations with the extracted entities.
5. Preserve the surface form used in the passage whenever possible.
6. Do not return an empty list.
7. Do not include explanations outside the JSON object.

Example passage:

Radio City is India's first private FM radio station and was started on 3 July 2001. It plays Hindi, English and regional songs. Radio City recently forayed into New Media in May 2008 with the launch of a music portal, PlanetRadiocity.com, that offers music-related news, videos, songs, and other features.

Example output:

```
{
  "entities": [
    "Radio City",
    "India",
    "private FM radio station",
    "3 July 2001",
    "Hindi",
```

```

    "English",
    "regional songs",
    "New Media",
    "May 2008",
    "PlanetRadiocity.com",
    "music portal",
    "music-related news",
    "videos",
    "songs"
  ]
}

```

Input:
 Passage: {passage}

A.1.2 Atomic-Fact Hyperedge Extraction

Task:
 Given a passage and a predefined list of entities, decompose the passage into atomic facts. Each atomic fact should express one self-contained claim and explicitly specify the entities involved in that fact.

Output format:
 Return only a valid JSON object with the following structure:

```

{
  "hyperedges": [
    {
      "text": "a complete atomic fact",
      "entities": ["entity_1", "entity_2",
      "..."]
    }
  ]
}

```

Guidelines:

1. Each hyperedge must contain exactly one factual claim or relation.
2. Each hyperedge must be understandable without requiring the original passage.
3. Use only entities from the provided entity list. Do not introduce new entities.
4. Make the relation among entities explicit and unambiguous.
5. Preserve temporal, causal, comparative, and modifier information when it affects the meaning of the fact.
6. When a statement involves more than two entities, keep them together in a single hyperedge rather than splitting the fact into disconnected binary relations.
7. Include all relevant entities from the provided entity list in both the hyperedge text and the entity array.
8. Ensure that the set of hyperedges covers all important factual information in the passage.
9. Do not include explanations outside the JSON object.

Example passage:
 In 2020, after Apple launched the M1 chip, major software companies like Adobe optimized their applications, improving performance by up to 80 percent compared to Intel-based Macs.

Entity list:

```

[
  "Apple",
  "M1 chip",

```

```

    "2020",
    "Adobe",
    "Adobe's applications",
    "Intel-based Macs",
    "80% performance improvement"
  ]
}

```

Example output:

```

{
  "hyperedges": [
    {
      "text": "Apple launched the M1 chip in 2020.",
      "entities": ["Apple", "M1 chip", "2020"]
    },
    {
      "text": "Adobe optimized Adobe's applications for the M1 chip after the chip was launched.",
      "entities": ["Adobe", "Adobe's applications", "M1 chip"]
    },
    {
      "text": "Adobe's applications on the M1 chip achieved up to an 80% performance improvement compared with Adobe's applications on Intel-based Macs.",
      "entities": [
        "Adobe",
        "Adobe's applications",
        "M1 chip",
        "80% performance improvement",
        "Intel-based Macs"
      ]
    }
  ]
}

```

Input:
 Passage: {passage}
 Entity list: {entities_json_list}

A.1.3 Question Decomposition

Task:
 Break the input question into the smallest number of retrieval-friendly sub-questions needed to answer it reliably.

Main principle:
 Decompose the question according to bridge facts, not according to surface length. Each sub-question should normally resolve one unknown bridge entity, one bridge relation, or the final requested attribute.

Guidelines:

1. If the question can be answered with a single retrieval step, return a list containing only the original question.
2. For sequential reasoning, create one sub-question for each bridge step when a later step depends on the answer to an earlier step.
3. A good sub-question should usually do one of the following:
 - identify a bridge entity;
 - identify a relation involving a known bridge entity;
 - ask for the final attribute once the target entity is known.

4. Avoid sub-questions that require resolving multiple new bridge entities at the same time.
5. Prefer two sub-questions for one-bridge questions, three for two-bridge questions, and four for three-bridge questions when needed.
6. Do not force every question into two sub-questions. Use more steps when this makes retrieval clearer.
7. Preserve the reasoning order. Later sub-questions may refer to earlier answers using phrases such as "that person", "that city", or "that work".
8. For parallel or comparative questions, retrieve the required attributes separately before asking for the comparison.
9. Avoid redundant or overlapping sub-questions.
10. Return at most four sub-questions.

Output format:

Return only a valid JSON object:

```
{
  "sub_questions": [
    "first sub-question",
    "second sub-question"
  ]
}
```

Examples:

Question:

What county is Erik Hort's birthplace a part of?

Output:

```
{
  "sub_questions": [
    "What is Erik Hort's birthplace?",
    "Which county is that birthplace part of?"
  ]
}
```

Question:

When was the start of the battle of the birthplace of the performer of III?

Output:

```
{
  "sub_questions": [
    "Who is the performer of III?",
    "What is that performer's birthplace?",
    "When did the battle at that birthplace start?"
  ]
}
```

Input:

Question: {question}

A.1.4 Sub-query Evaluation

Task:

Given a sub-query, retrieved atomic-fact hyperedges, and source passages, decide whether the evidence answers the sub-query.

Output format:

Return only a valid JSON object:

```
{
  "status": "ANSWERED" or "UNANSWERED",
  "answer": "short sub-answer or null",
  "supporting_hyperedge_ids": [0, 1]
}
```

Guidelines:

1. Use ANSWERED only when the hyperedges and passages support a direct answer.
2. The answer must be concise.
3. supporting_hyperedge_ids are 0-based indices from the listed hyperedges.
4. Do not include explanations outside the JSON object.

Sub-query: {sub_query}

Hyperedges:

{hyperedges_block}

Passages:

{passages_block}

A.1.5 Final Answer Integration

Task:

Given the original question and the sub-question answers produced during multi hop reasoning, combine the intermediate answers into a single final answer.

Guidelines:

1. The final answer must directly answer the original question.
2. Use the available sub-question answers.
3. If one sub-answer is missing, still provide the best short answer supported by the available information.
4. Output only the answer, without reasoning or restating the question.
5. Return only a valid JSON object.

Output format:

```
{
  "final_answer": "..."
}
```

Original question:

{original_question}

Sub-questions and answers:

{qa_block}

A.1.6 QA Reader Fallback

System prompt:

As an advanced reading comprehension assistant, your task is to analyze text passages and corresponding questions meticulously. Your response start after "Thought: ", where you will methodically break down the reasoning process, illustrating how you arrive at conclusions. Conclude with exactly one line formatted as "FINAL_ANSWER: <answer>". The final answer must be a concise answer span only, with no explanation, no restated question, and no extra text after it.

User prompt:

Wikipedia Title: {title_1}
{passage_text_1}

Wikipedia Title: {title_2}
{passage_text_2}

...

Retrieved propositions:

726

727

728

729

730

731

732

733

Dataset	Questions	Passages	Entities	Hyperedges
2WikiMultiHopQA	1,000	6,119	57,906	55,032
HotpotQA	1,000	9,812	112,308	99,394
MuSiQue	1,000	11,656	117,894	105,542

Table 6: Dataset statistics for the 1,000-question evaluation subsets.

{numbered_propositions}

Retriever hypothesis: {retriever_hypothesis}

Question: {question}

Thought:

A.1.7 Answer Judge

Task:

Evaluate whether a model prediction correctly answers the question by comparing it against the reference answer.

Decision rule:

Return [[yes]] if the prediction is factually equivalent to the reference answer or contains the reference answer. Return [[no]] otherwise.

Guidelines:

1. Accept paraphrases, aliases, abbreviations, and equivalent date formats.
2. Accept a response that contains the required answer even if it includes additional reasoning.
3. Return [[no]] if the response contains only part of the required answer.
4. Return [[no]] if the response is broader, narrower, or contradictory.
5. Return only [[yes]] or [[no]].

Question:

{question}

Reference answer:

{ground_truth}

Model prediction:

{prediction}

Is the model prediction correct?

A.2 Dataset Statistics

We evaluate HyperProve on three open-domain multi-hop question answering benchmarks: MuSiQue (Trivedi et al., 2022), HotpotQA (Yang et al., 2018), and 2WikiMultiHopQA (Ho et al., 2020). Following PropRAG (Wang and Han, 2025), we use the corresponding testing set for each dataset to ensure a consistent and comparable evaluation protocol. For the reported experiments, each benchmark is evaluated on a 1,000-question subset from its testing set, and the corpus passages associated with that split are processed offline to construct HyperProve’s atomic-fact hypergraph.

MuSiQue. MuSiQue is a multi-hop question answering benchmark designed to test compositional reasoning over multiple supporting passages. Questions in MuSiQue are constructed to reduce short-cut reasoning and require models to combine evidence from distinct reasoning steps before producing the final answer.

HotpotQA. HotpotQA is a widely used multi-hop question answering benchmark built from Wikipedia. It contains questions that require reasoning over multiple documents, including bridge-style questions, where an intermediate entity must be identified, and comparison questions, where information about multiple entities must be compared.

2WikiMultiHopQA. 2WikiMultiHopQA is an entity-centric multi-hop question answering benchmark constructed using Wikipedia and Wikidata. It emphasizes reasoning over structured entity relations and typically requires combining evidence across multiple linked facts or documents.

Table 6 summarizes the three datasets used in our experiments. These benchmarks cover complementary forms of multi-hop reasoning, including compositional reasoning, bridge reasoning, comparison reasoning, and entity-relation reasoning. In HyperProve, each extracted atomic fact is represented as one hyperedge connected to its participating entities and source passage.

A.3 Positioning Relative to Prior Work

Table 7 positions HyperProve relative to the evaluated baselines by comparing their retrieval units and multi-step control mechanisms. Dense retrievers rank passages in a single step, which makes them useful for seed retrieval but leaves them without an explicit cross-hop evidence state. Structured RAG systems add graph, fact-path, or hypergraph connectivity, but their expansion is primarily driven by static structure or pre-defined retrieval paths. Iterative systems introduce multi-step control, although their state is mainly textual or tied to passages, triples, or sentence-level evidence.

HyperProve combines these directions by retrieving over atomic-fact hyperedges, preserving n-ary entity context and source-passage provenance, and using intermediate answers to define the next local expansion frontier. This makes retrieval stateful, fact-centered, and structurally constrained while still allowing fresh dense seeds to recover evidence outside the carried frontier.

Method	Category	Retrieval Unit	Multi-step	Main Role
Contriever	Dense Retrieval	Passage	No	Unsupervised dense retrieval baseline
BGE-M3	Dense Retrieval	Passage	No	Strong dense retriever and HyperProve seed retriever
HippoRAG2	Structured RAG	Graph memory	Yes	Associative graph retrieval baseline
PropRAG	Structured RAG	Fine-grained fact path	Yes	Structured fact retrieval baseline
HyperGraphRAG	Structured RAG	Hypergraph fact	Yes	Hypergraph-structured retrieval baseline
IRCoT	Iterative Retrieval	Passage	Yes	Retrieval interleaved with reasoning
T2RAG	Iterative Retrieval	Triple	Yes	Triplet-driven retrieval and reasoning
HopRAG	Iterative Retrieval	Passage / evidence	Yes	Retrieve–reason–prune baseline
ChainRAG	Iterative Retrieval	Passage / sentence	Yes	Progressive retrieval baseline
HyperProve	Ours	Atomic-fact hyperedge	Yes	Answer-guided hypergraph expansion

Table 7: Summary of baseline methods and retrieval units. HyperProve differs from prior methods by combining atomic-fact hyperedges with answer-guided, stateful expansion over local hypergraph neighborhoods.

A.4 Case Study: Hidden-Bridge Reasoning

Figure 4 shows a hidden-bridge MuSiQue example where the question asks: “When was the start of the battle of the birthplace of the performer of *III*?” The original question does not explicitly mention the required bridge entities: *Stanton Moore*, *New Orleans*, or the *Battle of New Orleans*. This makes the example difficult for retrieval methods that mainly rank evidence according to surface similarity with the original question.

HyperProve decomposes the question into dependent sub-queries and carries intermediate answers forward as bridge entities. It first identifies *III* as Stanton Moore’s third studio solo album, then uses *Stanton Moore* to retrieve evidence about his birthplace, *New Orleans*. The system then uses *New Orleans* as the next bridge entity to retrieve evidence about the *Battle of New Orleans* and its start date. This answer-guided evidence chain leads to the correct answer, *December 14, 1814*.

HippoRAG2, by contrast, over-emphasizes broad surface terms from the original question, such as *III*, *battle*, and *birthplace*. Because the key intermediate entities are not present in the original question, its associative retrieval process follows unrelated evidence and fails to recover the hidden bridge chain. It therefore produces the incorrect answer, *1 September 1054*. This case highlights the importance of answer-conditioned bridge propagation: in multi-hop QA, the next useful evidence is often not lexically similar to the original question, but becomes reachable only after an intermediate entity has been resolved.

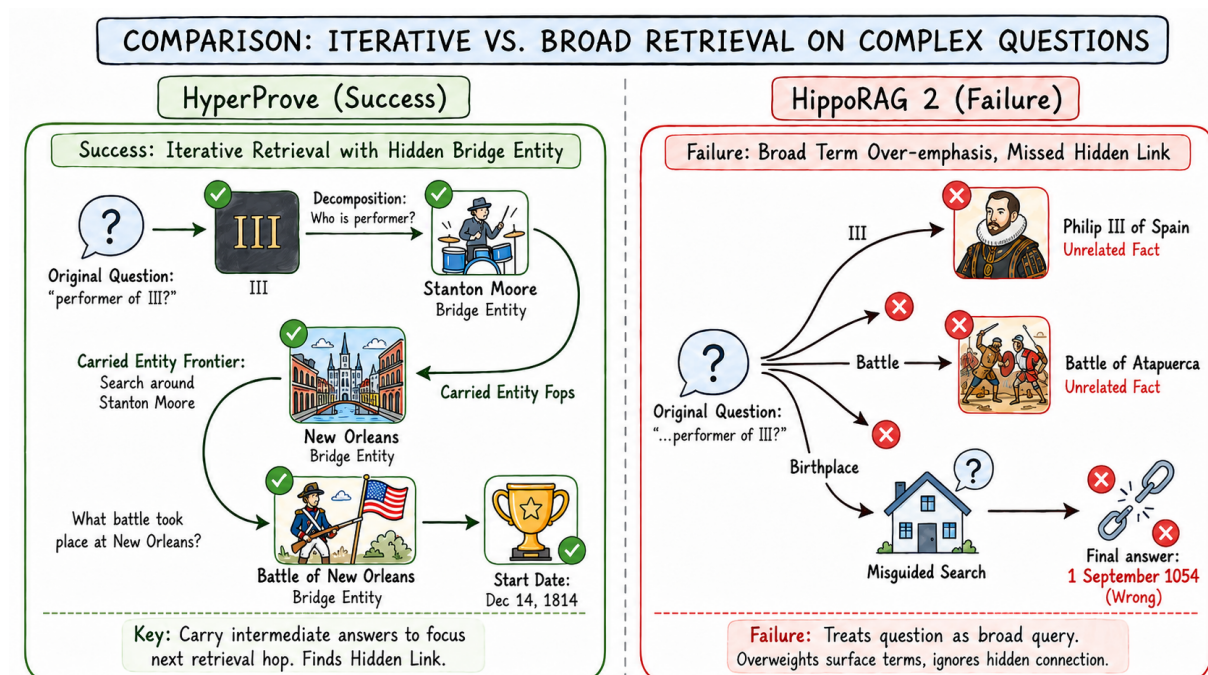


Figure 4: Qualitative comparison on a hidden-bridge MuSiQue example. HyperProve carries intermediate answers as bridge entities, enabling retrieval around *Stanton Moore*, *New Orleans*, and the *Battle of New Orleans*. In contrast, HippoRAG2 over-emphasizes broad surface terms from the original question and misses the hidden evidence chain.